CyberShield AI: A Web-Based Real-Time Cybersecurity Monitoring Dashboard with AI-Assisted Threat Analysis

Project Group 16

Sreedevi R P Roll No: 59
Vishnu V Nair Roll No: 65
Abhi Raj Roll No: 66

Under the Guidance of

Varsha C Mohan
Assistant Professor,Dept. of Computer Engineering

CSD415 Project Phase 1



Dept. of Computer Engineering
College of Engineering, Chengannur
APJ Abdul Kalam Technological University

Introduction



Introduction
Problem Statement
Literature Review
Proposed Solution
System Design
Architecture
Use Case
Data Flow
Methodology
Requirements
Results
Conclusion
Future Scope
References

- Phishing remains the primary initial-access vector (over 85% of breaches); modern malware evades detection with fileless payloads, packers and obfuscation.
- Small enterprises and junior SOC analysts cannot afford or operate fragmented enterprise security suites.

What is CyberShield AI?

A lightweight, web-based SOC platform unifying real-time telemetry, automated detection, threat intelligence and alert dispatch in one command center.

- **Dual-engine phishing detection:** cloud AI (235k URLs) + Random Forest.
- **Explainable AI:** Google Gemini gives plain-English reasoning.
- **Malware forensics:** client-side hashing, 72+ vendor correlation.
- **Live telemetry & alerting:** 10 s refresh, Discord/Slack webhooks.

Problem Statement



Introduction
Problem Statement
Literature Review
Proposed Solution
System Design
Architecture
Use Case
Data Flow
Methodology
Requirements
Results
Conclusion
Future Scope
References

To address fragmented cybersecurity tools and unexplained threat alerts by integrating dual-engine phishing detection, malware analysis, AI-based threat explanations, and real-time security alerting into a unified monitoring platform.

Literature Review (1/2)



Introduction
Problem Statement
Literature Review
Proposed Solution
System Design
Architecture
Use Case
Data Flow
Methodology
Requirements
Results
Conclusion
Future Scope
References

Paper	Methodology	Research Gap	CyberShield Solution
[1] Misiek & Hyla (2026) XGBoost URL Phishing Detector	Extract lexical URL features and train an XGBoost classifier. Evaluate generalisation across multiple datasets.	No real-time alert notification system.	Real-time Discord/Slack webhook alerts.
[2] Koide et al. (2026) ChatSpamDetector (LLM)	Decode and simplify emails, analyze them using an LLM, and generate verdicts with explanations.	Limited human-readable explanations for analysts.	Gemini provides explanations in plain English.
[3] Aldhuwayhi & Alromih (2026) Dynamic Phishing URL Detection	Combine machine-learning URL classification with crowdsourced threat intelligence.	Difficulty adapting to emerging phishing threats.	Crowdsourced threat verification loop.

Literature Review (2/2)

Introduction
Problem Statement
Literature Review
Proposed Solution
System Design
Architecture
Use Case
Data Flow
Methodology
Requirements
Results
Conclusion
Future Scope
References

Paper	Methodology	Research Gap	CyberShield Solution
[4] Duarte et al. (2025) Parked-Domain Phishing Detection	Extract features from parked and newly registered domains and use machine learning to detect phishing URLs.	Newly generated domains may evade blacklists.	15 URL features and Shannon entropy.
[5] Elgarduh et al. (2026) HFIM Multi-Channel Image Malware Detection	Represent structural, semantic, and behavioural malware features as RGB images and classify them using CNNs.	High computational cost limits browser-based use.	Client-side SHA-256 hashing and 72+ antivirus engines.
[6] Gutierrez et al. (2024) Deep-Learning Real-Time Malware Detection	Train deep-learning models on malware features and evaluate detection accuracy and inference speed.	High latency and interface performance bottlenecks.	Fast lexical analysis with asynchronous cloud AI.

Proposed Solution



Introduction
Problem Statement
Literature Review
Proposed Solution
System Design
Architecture
Use Case
Data Flow
Methodology
Requirements
Results
Conclusion
Future Scope
References

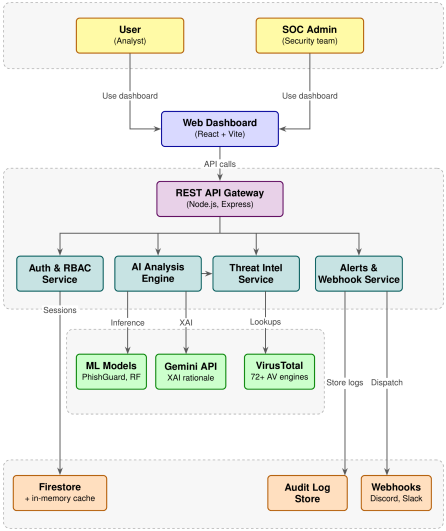
Vision: bridge ML detection and human response through a full-stack, browser-based platform.

1. **Dual-engine phishing detector:** Engine 1 PhishGuard Cloud AI (235k URLs); Engine 2 CyberShield Random Forest (15 features); a consensus engine reports full agreement or discordance.
2. **AI SOC Co-Pilot (Gemini):** turns lexical indicators and PE headers into 2-sentence triage summaries.
3. **In-browser forensics:** SHA-256 via Web Crypto; files are never uploaded.
4. **Threat intelligence:** CVEs, IP blacklists and IOCs with instant client-side search.
5. **Instant escalation:** configurable Discord/Slack webhooks.

System Architecture

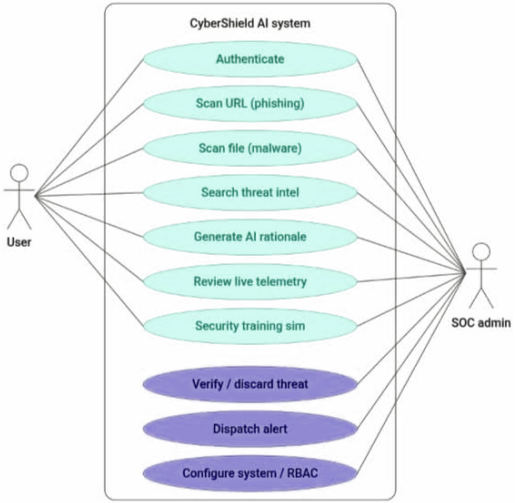


Introduction
Problem Statement
Literature Review
Proposed Solution
System Design
Architecture
Use Case
Data Flow
Methodology
Requirements
Results
Conclusion
Future Scope
References



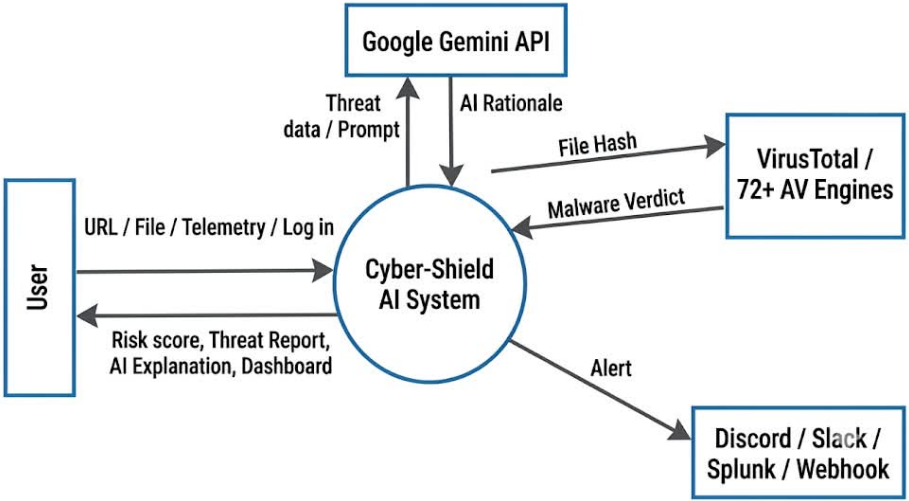
Use Case Diagram

Introduction
Problem Statement
Literature Review
Proposed Solution
System Design
Architecture
Use Case
Data Flow
Methodology
Requirements
Results
Conclusion
Future Scope
References



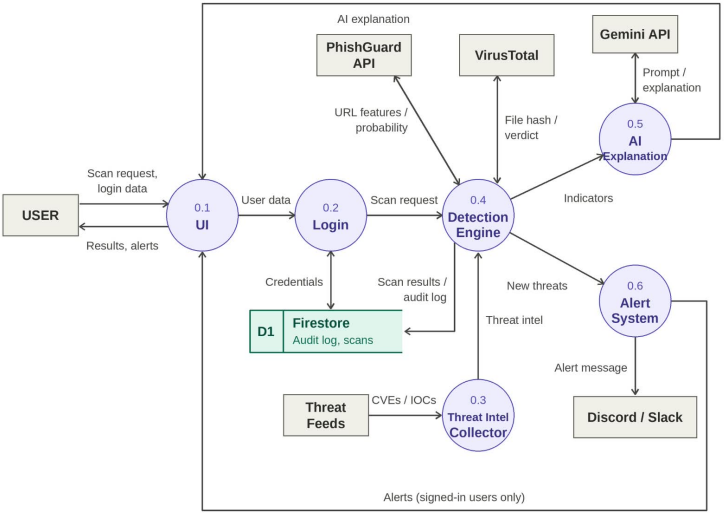
DFD Level 0: Context

Introduction
Problem Statement
Literature Review
Proposed Solution
System Design
Architecture
Use Case
Data Flow
Methodology
Requirements
Results
Conclusion
Future Scope
References



DFD Level 1: Decomposed

Introduction
Problem Statement
Literature Review
Proposed Solution
System Design
Architecture
Use Case
Data Flow
Methodology
Requirements
Results
Conclusion
Future Scope
References



Methodology: URL Feature Engineering



Introduction
Problem Statement
Literature Review
Proposed Solution
System Design
Architecture
Use Case
Data Flow
Methodology
Requirements
Results
Conclusion
Future Scope
References

15 core predictors

1. **Structure:** URL, hostname, path and query length.
2. **Tokens:** Counts of dots, hyphens, slashes, `@` and digits.
3. **Domain:** Subdomain count via `tldextract`.
4. **Keywords:** `login`, `verify`, `banking`, `secure`, `wallet`.
5. **Network markers:** IP-as-host, HTTPS validity and URL shorteners.
6. **Shannon entropy:** Detects obfuscation and DGA domains.

$$H(X) = -\sum_{i=1}^{n} P(x_i) \log_2 P(x_i)$$

Methodology: Model and Consensus



Introduction
Problem Statement
Literature Review
Proposed Solution
System Design
Architecture
Use Case
Data Flow
Methodology
Requirements
Results
Conclusion
Future Scope
References

1. **Classifier:** `RandomForestClassifier`, 100 estimators, Gini criterion, trained on a balanced legitimate/malicious URL corpus using `train_model.py`. Saved as `.joblib`.
2. **Consensus score:**

$$\text{Score}_{\text{comb}} = 0.5\,\text{Score}_{\text{PhishGuard}} + 0.5\,\text{Score}_{\text{CyberShield}}$$

3. `FULL_AGREEMENT` indicates high confidence, while `DISCORDANT` raises triage priority.
4. **Malware forensics:** SHA-256 hashing in the browser via `crypto.subtle.digest`. Byte entropy > 7.8 flags potentially packed or encrypted executables.

Functional Requirements



Introduction
Problem Statement
Literature Review
Proposed Solution
System Design
Architecture
Use Case
Data Flow
Methodology
Requirements
Results
Conclusion
Future Scope
References

- **Dashboard:** system status, threats blocked, anomalies, live telemetry.
- **Phishing analysis:** dual-engine verdict, risk 0–100, severity, Gemini rationale.
- **Malware forensics:** client-side SHA-256, PE headers, entropy, multi-vendor match.
- **Network telemetry:** live events, egress spikes, Tor exit-node flags.
- **Threat intel:** CVEs, malicious IPs, IOCs with sub-second filtering.
- **Alerting:** dedupe, LOW to CRITICAL severity, webhook push.
- **HITL & RBAC:** crowdsourced verification; Admin vs Analyst roles.

Non-Functional Requirements



Introduction
Problem Statement
Literature Review
Proposed Solution
System Design
Architecture
Use Case
Data Flow
Methodology
Requirements
Results
Conclusion
Future Scope
References

- **Performance:** hashing under 50 ms; dual-engine + AI explanation under 2.5 s.
- **Scalability:** modular services; add models or feed parsers independently.
- **Security & privacy:** files hashed locally and never leave the browser unauthorised; RBAC guards admin actions.
- **Usability:** dark-mode SOC design; green = safe, yellow = suspicious, red = phishing/critical.
- **Reliability:** fallbacks for API cold starts, in-memory cache, resilient Firestore persistence.

Hardware Requirements



Introduction
Problem Statement
Literature Review
Proposed Solution
System Design
Architecture
Use Case
Data Flow
Methodology
Requirements
Results
Conclusion
Future Scope
References

Development & training

- CPU: Core i5/i7 (8th gen+), Ryzen 5 or Apple Silicon.
- RAM: 8 GB minimum (16 GB recommended); 256 GB SSD with 5 GB free.
- Network: 10 Mbps+ for external API calls.

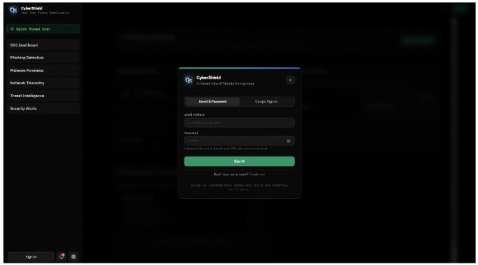
Software Requirements

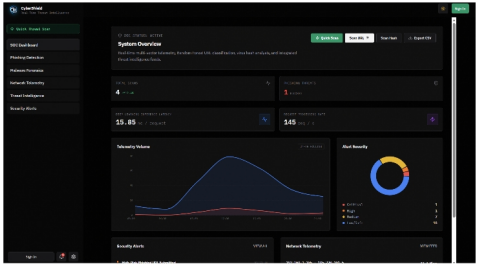


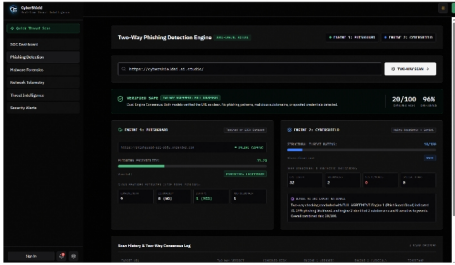
Introduction
Problem Statement
Literature Review
Proposed Solution
System Design
Architecture
Use Case
Data Flow
Methodology
Requirements
Results
Conclusion
Future Scope
References

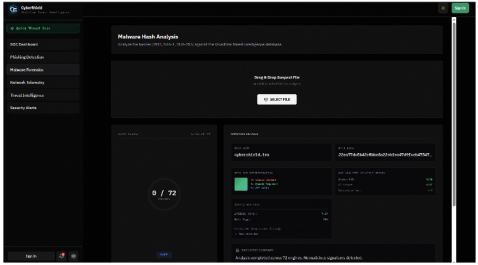
Operating System	Ubuntu 22.04 LTS, macOS, Windows 10/11
Frontend	React 18, TypeScript, Vite 6, Tailwind CSS v4, Recharts, Lucide React
Backend	Node.js 18/20 LTS, Express.js 4, TypeScript (`tsx`), Axios
ML / AI	Python 3.10+, Scikit-learn, Pandas, NumPy, Joblib, `tldextract`, Google Gemini SDK (`@google/genai`)
Cloud Services	Firebase Firestore, Cloud Run, Render, Discord and Slack webhooks

Results



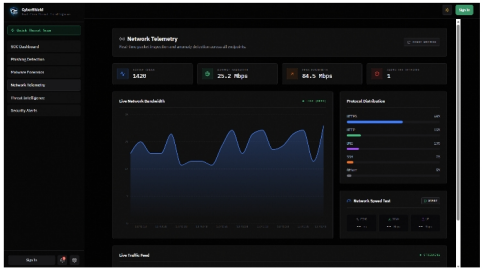


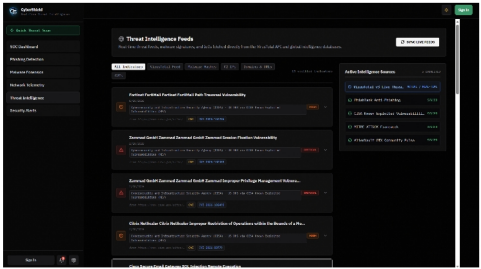


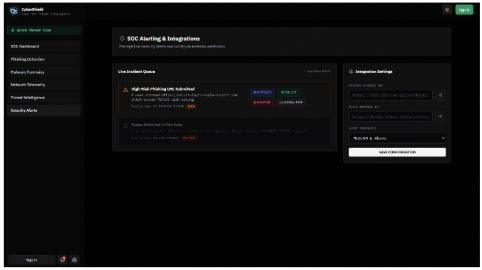


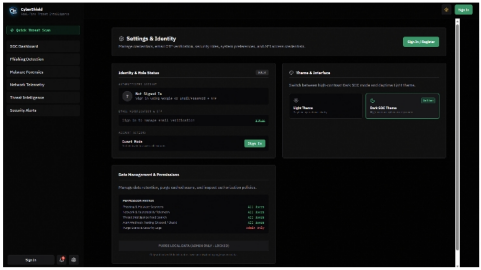
Introduction
Problem Statement
Literature Review
Proposed Solution
System Design
Architecture
Use Case
Data Flow
Methodology
Requirements
Results
Conclusion
Future Scope
References

Results









Introduction
Problem Statement
Literature Review
Proposed Solution
System Design
Architecture
Use Case
Data Flow
Methodology
Requirements
Results
Conclusion
Future Scope
References

Conclusion



Introduction
Problem Statement
Literature Review
Proposed Solution
System Design
Architecture
Use Case
Data Flow
Methodology
Requirements
Results
Conclusion
Future Scope
References

- CyberShield AI bridges academic ML research and real SOC operations.
- Dual-engine cross-verification, Gemini explanations, client-side malware analysis and webhook alerting form one cohesive defence tool.
- Small IT teams can deploy explainable threat detection without heavy enterprise cost.

Future Scope



Introduction
Problem Statement
Literature Review
Proposed Solution
System Design
Architecture
Use Case
Data Flow
Methodology
Requirements
Results
Conclusion
Future Scope
References

1. **Sandbox execution:** automated browser sandboxing to detonate malicious URLs and zero-day scripts.
2. **Live feed scraping:** continuous ingestion from PhishTank, OpenPhish and URLhaus.
3. **YARA & Sigma compiler:** custom rules for local file classification.
4. **Mobile & PWA:** push notifications to incident responders.

References I



Introduction
Problem Statement
Literature Review
Proposed Solution
System Design
Architecture
Use Case
Data Flow
Methodology
Requirements
Results
Conclusion
Future Scope
References

1. M. Misiek and T. Hyla, "XGBoost-Based URL Phishing Detection Method With Cross-Dataset Validation," *IEEE Access*, vol. 14, 2026.
2. T. Koide, H. Nakano, and D. Chiba, "ChatSpamDetector: Large Language Model-Based Phishing Email Detection," *IEEE Access*, vol. 14, 2026.
3. L. Aldhuwayhi and A. Alromih, "A Dynamic Phishing URL Detection System Integrating Advanced Machine Learning and Crowdsourced Threat Intelligence," *IEEE Access*, 2026.
4. J. D. Duarte et al., "Machine Learning for Early Detection of Phishing URLs in Parked Domains," *IEEE Access*, vol. 13, 2025.
5. A. Elgarduh et al., "Hybrid Feature Image Mapping (HFIM): A Multi-Channel Image Approach for Malware Detection," *IEEE Access*, 2026.
6. R. Gutierrez et al., "Application of Deep Learning Models for Real-Time Automatic Malware Detection," *IEEE Access*, vol. 12, 2024.

The End